



南開大學
Nankai University

计算机学院
深度学习实验报告

2D 扩散模型实验报告

姓名：王一诺
学号：2312331
专业：计算机科学与技术

2026 年 6 月 14 日

目录

1	实验要求	2
2	扩散模型基本代码	2
3	三种模型训练目标	5
4	模型结构与训练轮数调整实验	7
5	对比分析	10
5.1	不同训练目标的差异	10
5.2	模型结构和训练轮次的影响	10
5.3	指标解释	10
6	实验心得	11

1 实验要求

- 掌握扩散模型的基本原理;
- 学会使用 PyTorch 搭建简单的 MLP 网络;
- 学会使用 PyTorch 实现扩散模型的训练和采样代码;
- 学会使用 PyTorch 运行扩散模型的训练和测试, 并得到生成的数据和指标结果。
- 实现三种不同训练目标 (噪声 ε 、干净样本 x_0 、速度 v), 并比较其生成效果;
- 尝试修改 MLP 网络的宽度、深度以及训练轮次, 观察其对生成质量的影响。

2 扩散模型基本代码

本实验在二维螺旋数据上实现扩散模型。数据生成函数使用极坐标螺旋曲线, 并将坐标归一化到近似 $[-1, 1]$ 范围内。前向扩散采用线性噪声表, 时间步数 $T = 1000$, β_t 从 10^{-4} 线性增加到 0.02。

Listing 1: 数据生成与扩散噪声表

```
1 def make_spiral(n=2000):
2     theta = torch.linspace(0, 4 * math.pi, n)
3     r = theta
4     x = r * torch.cos(theta)
5     y = r * torch.sin(theta)
6     data = torch.stack([x, y], dim=1)
7     data = data / data.abs().max()
8     return data
9
10 T = 1000
11 betas = torch.linspace(1e-4, 0.02, T).to(device)
12 alphas = 1.0 - betas
13 alphas_cumprod = torch.cumprod(alphas, dim=0)
14
```

模型输入为带噪二维点 $x_t = (x, y)$ 和时间步 t , 其中 t 先归一化到 $[0, 1]$, 再与二维坐标拼接, 因此 MLP 实际输入维度为 3。模型输出维度为 2, 可根据训练目标解释为噪声、干净样本或速度。

Listing 2: 基础 MLP 模型结构

```
1 class Model(nn.Module):
2     def __init__(self):
3         super().__init__()
4         # 5层MLP: 输入为(x, y, t), 输出仍为2维向量
5         self.net = nn.Sequential(
6             nn.Linear(3, 256),
7             nn.SiLU(),
8             nn.Linear(256, 256),
9             nn.SiLU(),
10            nn.Linear(256, 256),

```

```

11         nn.SiLU(),
12         nn.Linear(256, 256),
13         nn.SiLU(),
14         nn.Linear(256, 2),
15     )
16
17     def forward(self, x, t):
18         # 将时间步归一化到[0, 1], 再与二维坐标拼接
19         if t.dim() == 1:
20             t = t.unsqueeze(1)
21         t = t.float() / (T - 1)
22         inp = torch.cat([x, t], dim=1)
23         return self.net(inp)
24

```

前向加噪公式为

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon,$$

其中 $\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$ 。训练时随机采样真实点、时间步和标准高斯噪声，根据不同训练目标构造监督信号。

Listing 3: 训练函数核心代码

```

1 def train(model, mode='eps', epochs=10000):
2     data = make_spiral(5000).to(device)
3     optimizer = optim.Adam(model.parameters(), lr=1e-3)
4
5     for epoch in range(epochs):
6         idx = torch.randint(0, data.shape[0], (256,), device=device)
7         x0 = data[idx]
8
9         t = torch.randint(0, T, (256,), device=device)
10        alpha_bar = alphas_cumprod[t].unsqueeze(1)
11
12        noise = torch.randn_like(x0)
13        xt = torch.sqrt(alpha_bar) * x0 + torch.sqrt(1 - alpha_bar) * noise
14
15        # TODO: prediction
16        pred = model(xt, t)
17
18        # TODO: target based on mode
19        if mode == 'eps':
20            target = noise
21        elif mode == 'x0':
22            target = x0
23        elif mode == 'v':
24            # v = sqrt(alpha_bar) * eps - sqrt(1-alpha_bar) * x0
25            target = torch.sqrt(alpha_bar) * noise - torch.sqrt(1 - alpha_bar) * x0
26        else:
27            raise ValueError("mode must be one of 'eps', 'x0', or 'v'")

```

```

28
29     loss = ((pred - target) ** 2).mean()
30
31     optimizer.zero_grad()
32     loss.backward()
33     optimizer.step()
34
35     return model
36

```

采样阶段从标准高斯分布初始化 x_T ，随后从 $T - 1$ 反向迭代到 0。对于 x_0 预测和 v 预测，先转换回噪声预测，再使用 DDPM 形式的反向更新。

Listing 4: 采样函数核心代码

```

1 @torch.no_grad()
2 def sample(model, mode='eps', n=1000):
3     x = torch.randn(n, 2).to(device)
4
5     for t in reversed(range(T)):
6         # TODO: model prediction
7         t_batch = torch.full((n,), t, device=device, dtype=torch.long)
8         pred = model(x, t_batch)
9
10        # TODO: convert to noise
11        alpha_bar = alphas_cumprod[t]
12        if mode == 'eps':
13            pred_noise = pred
14        elif mode == 'x0':
15            pred_noise = (x - torch.sqrt(alpha_bar) * pred) / torch.sqrt(1 - alpha_bar)
16        elif mode == 'v':
17            # 由v预测量还原噪声: eps = sqrt(1-alpha_bar)*x_t + sqrt(alpha_bar)*v
18            pred_noise = torch.sqrt(1 - alpha_bar) * x + torch.sqrt(alpha_bar) * pred
19        else:
20            raise ValueError("mode must be one of 'eps', 'x0', or 'v'")
21
22        # TODO: update step
23        beta = betas[t]
24        alpha = alphas[t]
25        # DDPM反向一步: t>0时加入随机噪声, t=0时只保留均值
26        z = torch.randn_like(x) if t > 0 else torch.zeros_like(x)
27        x = (x - beta / torch.sqrt(1 - alpha_bar) * pred_noise) / torch.sqrt(alpha) +
28            torch.sqrt(beta) * z
29
30    return x.cpu()

```

整体训练与采样流程可以概括为如下伪代码。

Listing 5: 训练与采样伪代码

训练阶段：

1. 生成真实螺旋点 x_0
2. 随机采样时间步 t 和噪声 ϵ
3. 根据 $x_t = \sqrt{\alpha_t} * x_0 + \sqrt{1 - \alpha_t} * \epsilon$ 得到带噪样本
4. 将 (x_t, t) 输入 MLP
5. 根据 mode 选择监督目标 ϵ 、 x_0 或 v
6. 使用均方误差反向传播更新模型

采样阶段：

1. 从标准高斯分布采样 x_T
2. 对 $t = T-1, \dots, 0$ 反向迭代
3. 使用模型预测当前时间步的目标量
4. 将预测结果转换为噪声 pred_noise
5. 根据 DDPM 反向更新公式得到 x_{t-1}
6. 返回最终生成点云

3 三种模型训练目标

本实验分别测试了三种训练目标：

- **eps**: 模型直接预测前向扩散中加入的噪声 ϵ ;
- **x0**: 模型直接预测原始干净样本 x_0 ;
- **v**: 模型预测速度变量 $v = \sqrt{\alpha_t}\epsilon - \sqrt{1 - \alpha_t}x_0$ 。

评价指标使用 MMD 和 Chamfer Distance。MMD 衡量真实点云与生成点云的整体分布差异，Chamfer Distance 衡量两组点之间的最近邻匹配距离；两者均越小越好。

Listing 6: 评价指标代码

```

1 def compute_mmd(x, y, sigma=0.1):
2     def kernel(a, b):
3         dist = ((a[:, None, :] - b[None, :, :]) ** 2).sum(-1)
4         return torch.exp(-dist / (2 * sigma ** 2))
5     return kernel(x, x).mean() + kernel(y, y).mean() - 2 * kernel(x, y).mean()
6
7 def chamfer_distance(x, y):
8     dist = torch.cdist(x, y)
9     return dist.min(dim=1)[0].mean() + dist.min(dim=0)[0].mean()
10

```

三种训练目标均使用基础模型结构和 10000 轮训练，notebook 单元格输出如下。

Listing 7: eps 目标单元格输出

```

MMD: 0.02779237926006317
Chamfer: 0.1343379020690918

```

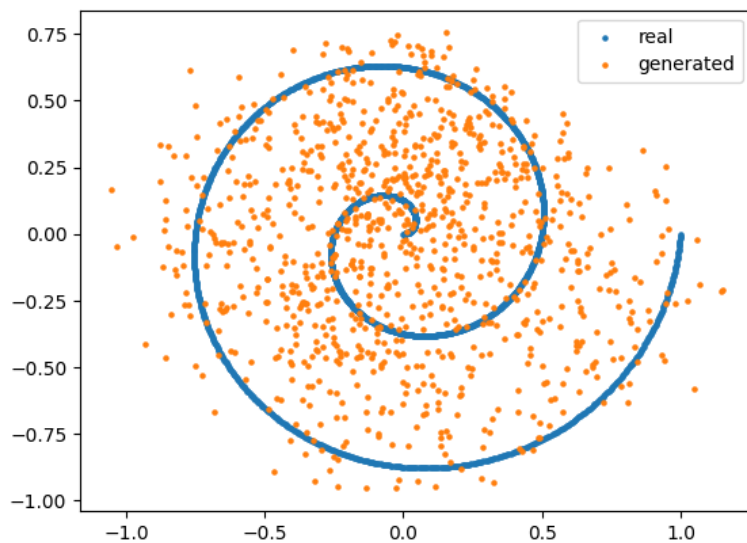


图 3.1: eps 目标下的真实点云与生成点云对比

Listing 8: x0 目标单元格输出

```
MMD: 0.03979836404323578  
Chamfer: 0.17328613996505737
```

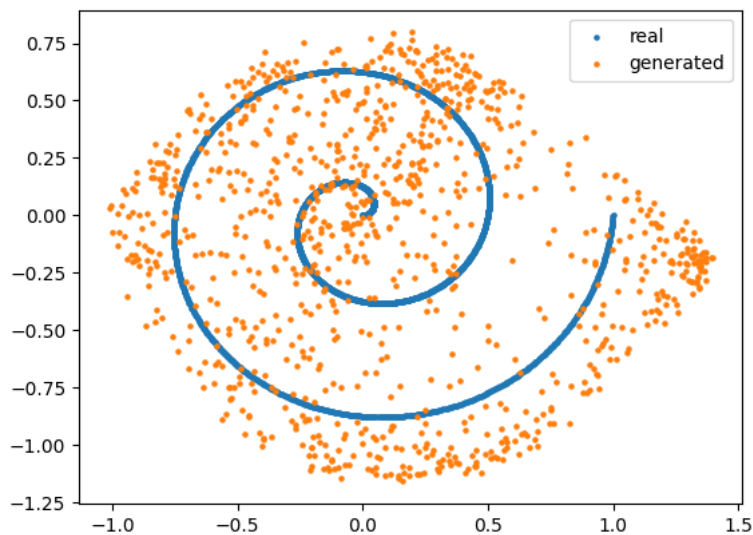


图 3.2: x0 目标下的真实点云与生成点云对比

Listing 9: v 目标单元格输出

```
MMD: 0.021081842482089996  
Chamfer: 0.1110987663269043
```

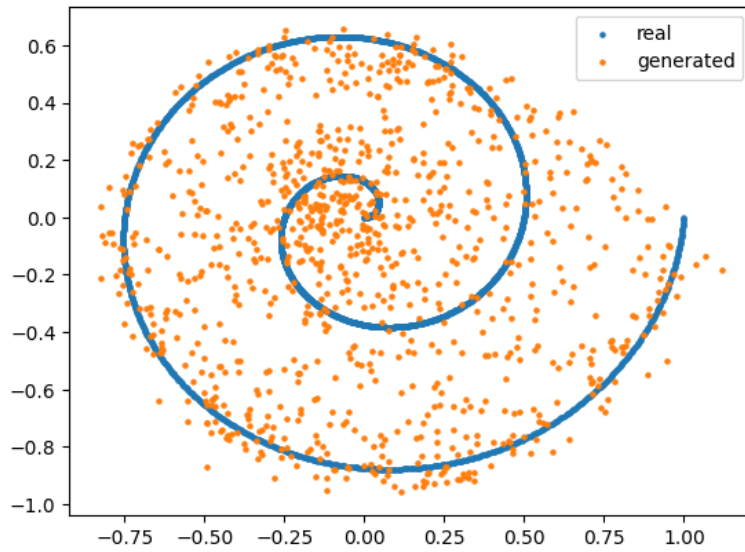
图 3.3: v 目标下的真实点云与生成点云对比

表 1: 三种训练目标结果对比

训练目标	训练轮次	MMD	Chamfer Distance
eps	10000	0.027792	0.134338
x0	10000	0.039798	0.173286
v	10000	0.021082	0.111099

从指标看，基础配置下 v 目标的 MMD 和 Chamfer Distance 最低，eps 目标次之，x0 目标最差。虽然三种方法均能生成一定范围内的二维点云，但与目标螺旋曲线仍存在明显差距，说明基础网络容量和训练轮次不足以稳定恢复完整螺旋结构。

4 模型结构与训练轮数调整实验

根据上一节结果， v 目标在基础设置下表现相对较好，因此在结构和轮次实验中固定使用 `mode='v'`，分别改变 MLP 宽度、深度和训练轮次。

Listing 10: 可调宽度与深度的 MLP

```

1 class MLPModel(nn.Module):
2     def __init__(self, hidden_dim=256, depth=4):
3         super().__init__()
4         layers = []
5         in_dim = 3 # 输入为(x, y, t)
6
7         for _ in range(depth):
8             layers.append(nn.Linear(in_dim, hidden_dim))
9             layers.append(nn.SiLU())

```



```

10         in_dim = hidden_dim
11
12         layers.append(nn.Linear(hidden_dim, 2))
13         self.net = nn.Sequential(*layers)
14
15     def forward(self, x, t):
16         # 与基础模型保持一致：时间步归一化后拼接到二维坐标后面
17         if t.dim() == 1:
18             t = t.unsqueeze(1)
19         t = t.float() / (T - 1)
20         inp = torch.cat([x, t], dim=1)
21         return self.net(inp)
22

```

Listing 11: 结构与轮次实验代码

```

1 experiment_mode = 'v'
2
3 experiment_configs = [
4     {'hidden_dim': 128, 'depth': 4, 'epochs': 10000}, # 更窄
5     {'hidden_dim': 256, 'depth': 4, 'epochs': 10000}, # 基准宽度
6     {'hidden_dim': 512, 'depth': 4, 'epochs': 10000}, # 更宽
7     {'hidden_dim': 256, 'depth': 6, 'epochs': 10000}, # 更深
8     {'hidden_dim': 256, 'depth': 4, 'epochs': 30000}, # 更多训练轮次
9 ]
10
11 experiment_results = []
12 for cfg in experiment_configs:
13     print(f"Running hidden={cfg['hidden_dim']}, depth={cfg['depth']}, epochs={cfg['epochs']},
14           mode={experiment_mode}")
15     result = evaluate_setting(**cfg, mode=experiment_mode)
16     experiment_results.append(result)
17     print(f"    MMD={result['mmd']:.6f}, Chamfer={result['chamfer']:.6f}")

```

notebook 中该实验的输出如下。

Listing 12: 结构与训练轮次实验单元格输出

```

Running hidden=128, depth=4, epochs=10000, mode=v
    MMD=0.033268, Chamfer=0.140384
Running hidden=256, depth=4, epochs=10000, mode=v
    MMD=0.013229, Chamfer=0.094650
Running hidden=512, depth=4, epochs=10000, mode=v
    MMD=0.006006, Chamfer=0.060511
Running hidden=256, depth=6, epochs=10000, mode=v
    MMD=0.006664, Chamfer=0.058992
Running hidden=256, depth=4, epochs=30000, mode=v
    MMD=0.004755, Chamfer=0.032316

```

Summary:

hidden	depth	epochs	mode	MMD	Chamfer
128	4	10000	v	0.033268	0.140384
256	4	10000	v	0.013229	0.094650
512	4	10000	v	0.006006	0.060511
256	6	10000	v	0.006664	0.058992
256	4	30000	v	0.004755	0.032316

表 2: MLP 宽度、深度和训练轮次实验结果

隐藏层宽度	隐藏层数	训练轮次	MMD	Chamfer Distance
128	4	10000	0.033268	0.140384
256	4	10000	0.013229	0.094650
512	4	10000	0.006006	0.060511
256	6	10000	0.006664	0.058992
256	4	30000	0.004755	0.032316

最佳配置为隐藏层宽度 256、隐藏层数 4、训练轮次 30000，单元格输出如下。

Listing 13: 最佳配置单元格输出

```
Best setting: {'hidden_dim': 256, 'depth': 4, 'epochs': 30000, 'mode': 'v', 'mmd':
0.004754558205604553, 'chamfer': 0.03231602907180786}
```

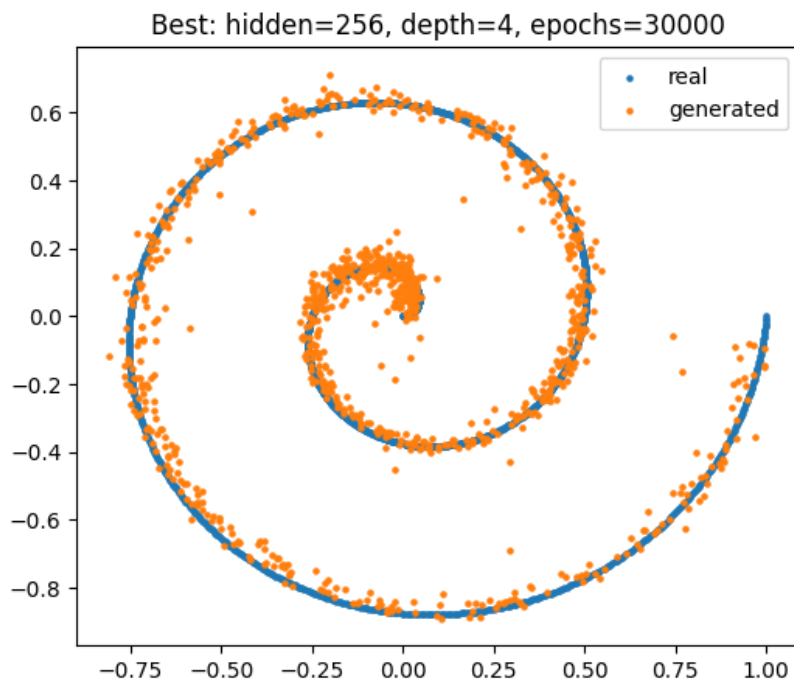


图 4.4: 结构与轮次实验中的最佳生成结果

可以看到，增加模型宽度、增加深度以及延长训练轮次均能显著改善指标。其中从宽度 128 提升

到 256 时, MMD 从 0.033268 下降到 0.013229, Chamfer 从 0.140384 下降到 0.094650; 继续提升到宽度 512 后, MMD 接近 0.006。增加深度到 6 层也明显改善了指标。最明显的提升来自训练轮次增加到 30000, 此时 MMD 为 0.004755, Chamfer 为 0.032316, 均达到 notebook 中给出的参考目标。

5 对比分析

5.1 不同训练目标的差异

在基础模型和 10000 轮训练条件下, 三种训练目标的表现 v 优于 eps , eps 优于 x_0 。其原因可以从监督信号的难度解释。

eps 预测目标直接学习前向扩散加入的高斯噪声, 与 DDPM 采样公式最直接匹配, 因此通常具有较好的稳定性。但是在本实验中, 基础模型只使用一个归一化时间标量作为时间条件, 表达能力有限, 因此 eps 目标虽然能够生成集中在数据范围附近的点云, 但很难完整恢复螺旋形状。

x_0 预测目标要求模型从不同噪声强度的 x_t 中直接恢复干净样本。尤其在大时间步时, x_t 已经接近高斯噪声, 原始二维坐标信息被严重破坏, 此时直接回归 x_0 难度较高。因此本实验中 x_0 目标的 MMD 和 Chamfer Distance 均最大, 生成点云与真实螺旋的贴合程度最差。

v 目标是噪声和干净样本的线性组合, 在不同噪声强度下具有更平衡的尺度。实验中 v 目标在基础配置下取得最低 MMD 和 Chamfer Distance, 说明它在本任务中比直接预测 x_0 更容易优化, 也比 eps 目标略稳定。

5.2 模型结构和训练轮次的影响

从结构实验可见, 模型容量对二维扩散模型效果影响明显。宽度 128 时模型拟合能力不足, 指标最差; 宽度提高到 256 后指标明显下降; 宽度 512 进一步降低 MMD, 但 Chamfer 仍高于目标阈值。这说明更宽的网络可以更好拟合不同时间步下的去噪函数。

增加深度到 6 层时, MMD 和 Chamfer 也显著优于基础配置, 说明更深的 MLP 可以表达更复杂的非线性去噪映射。不过深度增加后训练时间也会变长, 且过深网络可能需要更细致的学习率和初始化设置。

训练轮次的影响最为直接。隐藏层宽度 256、深度 4 的模型从 10000 轮增加到 30000 轮后, MMD 下降到 0.004755, Chamfer 下降到 0.032316, 达到目标要求。这表明基础模型结构本身具有足够表达能力, 但 10000 轮训练时尚未充分收敛; 延长训练可以显著改善螺旋曲线的覆盖程度和点云贴合程度。

5.3 指标解释

MMD 关注整体分布相似度。当生成点云覆盖了真实螺旋的大致范围时, MMD 会下降。Chamfer Distance 更关注点到曲线的局部几何贴合, 如果生成点云分散、断裂或偏离螺旋轨迹, Chamfer 会明显变大。因此在实验中, 部分配置的 MMD 已经接近目标, 但 Chamfer 仍偏高, 说明生成分布整体范围接近真实数据, 但局部形状还没有完全贴合螺旋曲线。

综合来看, v 目标配合更充分的训练轮次能够取得最好结果。最终最佳配置不仅 MMD 低于 0.006, Chamfer 也低于 0.04, 说明生成点云在整体分布和局部几何距离上均接近真实二维螺旋数据。

6 实验心得

本次实验完整实现了二维扩散模型中的模型结构、训练过程和采样过程。通过实现 ϵ 、 x_0 和 v 三种训练目标，可以直观看到不同监督目标会影响去噪网络的优化难度和最终生成质量。实验结果表明，在该二维螺旋数据任务中，直接预测 x_0 最不稳定， v 目标整体表现最好。

此外，实验也说明扩散模型的效果不仅取决于采样公式和训练目标，还明显受网络容量与训练轮次影响。基础模型在 10000 轮时效果不够理想，但通过增加训练轮次到 30000 后，指标达到实验目标。这说明对于扩散模型而言，去噪函数需要在大量随机时间步和噪声强度下反复学习，训练是否充分会直接决定生成样本质量。

通过本实验，我加深了对前向加噪、反向去噪、训练目标转换和生成质量评价指标的理解，也熟悉了使用 PyTorch 搭建简单 MLP 扩散模型的完整流程。